

AGENDA

발표 순서

1 서비스 개요

2 시장 경쟁력

3 핵심 기술

4 성능 및 경제성

5 신뢰성 및 비전

PROBLEM DEFINITION

정보의 홍수 속, 정작 나를 위한 학습은 없다

기존 교육 시장은 방대한 콘텐츠를 제공하지만,
개별 학습자의 **이해도를 고려한 맞춤형 자료** 생성에는
막대한 비용과 시간이 소요되는 한계가 있습니다.

MARKET COMPETITIVENESS

단편적인 기능을 넘어, 학습의 전 과정을 혁신합니다

핵심 기능	범용 LLM	인강·MOOC	Khanmigo	NotebookLM	일과 (ilgwa)
맞춤형 학습	○	X	○	○	○
구조화된 커리큘럼	X	○	○	X	○
전과정 자동화	X	X	X	X	○
복습·약점 추적	X	X	△	X	○
비용 효율성	○	X	△	○	○

"일과는 단순한 튜터가 아닌, 생성-학습-평가-분석을 연결하는 유일한 시스템입니다."

SOLUTION OVERVIEW

학습의 모든 과정을 AI가 설계하고 자동화합니다

개인화 커리큘럼

사용자의 학습 자료를 분석하여
최적화된 학습 경로를 AI가 즉시 설계합니다.

실시간 문제 생성

학습 내용에 기반한 **맞춤형 모의고사**를 생성하여
실전 감각을 극대화합니다.

지능형 피드백

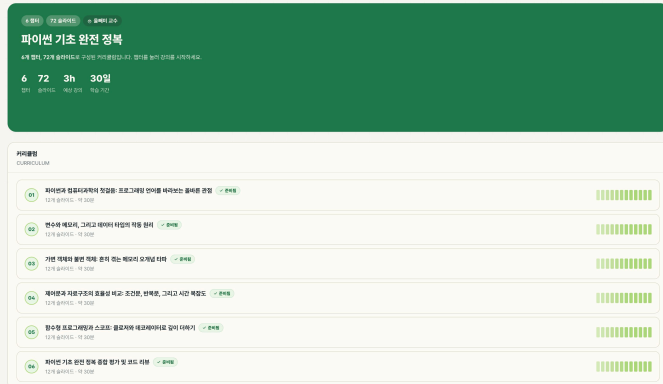
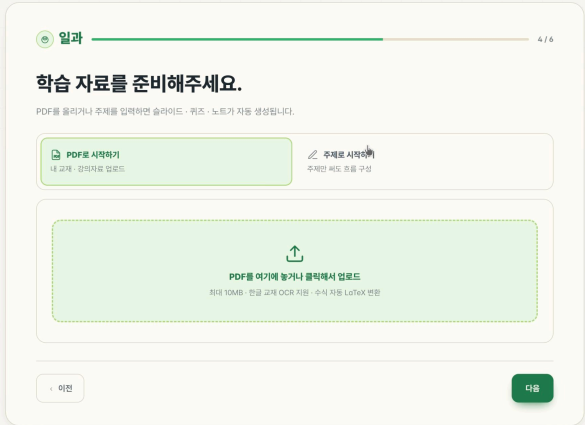
오답 분석을 통해 **취약 단원을 파악**하고
보완 자료를 자동 제공합니다.

학습 일관성 유지

검증된 템플릿을 통해 **항상 높은 품질**의
학습 콘텐츠를 보장합니다.

LIVE DEMONSTRATION

자료 업로드부터 생성까지, 단 10분 만에 완성되는 학습



1
자료 업로드

2 AI 분석

3 커리큘럼 생성

4 실시간 학습

+ 새 과외

2

- 강의별 자량 8강 · 1개

III. AI노트

ai_auto ☆

JVM을 벗어나 비어메탈로: Java 개발자가 바라보는 Rust의 철학 핵심 노트

JVM과 베어메탈의 철학적 차이

자바와 가장 머신은 이식성과 개발 편의성을 극대화하기 위해 하드웨어와 애플리케이션 사이에 추상화 계층을 뒀다.

리스트는 가장 머신 없이 하드웨어를 직접 제어하여 실행 속도를 높이고 메모리 사용량을 최소화하는 백터메탈 방식을 지칭한다.

이러한 철학적 차이는 개발자와 메모리 관리의 주도권을 가질 것인지 아니면 가장 머신에 위임할 것인지의 선택으로 이어진다.

소유권과 대여 규칙의 핵심 원리

리스트와 소수점은 모든 데이터에 한 하나의 명확한 주인을 지칭하여 이중 해지나 메모리 누수 문제를 원천적으로 방지합니다.

대여 규칙은 동시에 여러 개의 월기 전용 참조를 허용하거나 단 하나의 쓰기 가능한 참조만 허용하여 데이터 일관성을 유지합니다.

컴파일러의 빌드오류 검사기는 이러한 규칙들을 빌드 시점에 엄격하게 검증하여 런타임 오류가 발생할 가능성을 사전에 차단합니다.

자바 개발자가 흔히 하는 오해 바로잡기

러스트에 garbage 컬렉터가 없다고 해서 개발자가 C언어처럼 매번 수동으로 메모리를 해제해야 하는 것은 결코 아닙니다. 소유권을 가진 변수가 자신의 범위를 벗어나는 순간 컴파일러가 자동으로 메모리 해제 코드를 삽입해 주므로 안전합니다.

자바의 참조 타입과 달리 러스트의 대입 연산은 소유권 이전을 의미하므로 기존 변수를 다시 사용할 수 없음을 유의해야 합니다.

실전 전환을 위한 복습 루틴

자바 코드를 러스트로 변환할 때는 객체 간의 참조 관계를 단순화하고 소유권의 흐름을 명확하게 설계하는 연습이 필요합니다.

컴파일 여러 메시지를 두려워하지 말고 버러기로 검사가 제안하는 해결책을 꼼꼼히 읽으며 규칙을 체득하는 것이 좋습니다.

간단한 명령을 도구나 유틸리티 라이브러리를 직접 작성해 보면서 매번 조금씩 배우는 습관을 들여야 하는 시점을 조기에 보시기 바랍니다.

자동 생성된 강의안 + 음성 강의(TTS)

비용 없는 추상화의 실현

러스트의 또 다른 강력한 철학은 바로 비용 없는 추상화입니다.

이는 고수준의 편리한 추상화 도구들을 사용하더라도 컴파일러가 이를 최적화하여 직접 손으로 쓴 저수준 코드와 동일한 성능의 기계어를 만들어내는 것을 의미합니다.

자바의 제네릭이 런타임에 타입 소거를 거치며 박싱 오버헤드를 유발하는 것과 대조적입니다.

아래의 다이어그램은 러스트의 제네릭이 컴파일 타임에 단형화 과정을 거쳐 최적화되는 구조를 보여줍니다.



퀴즈

리스트의 '비용 없는 추상화(Zero-cost Abstractions)'가 의미하는 바는 무엇인가요?

- | | |
|--|--|
| 1. 러스트 라이브러리를 사용할 때 라이선스 비용이 전혀 들지 않는다 | 2. 교수님 코드를 작성하더라도 컴파일러 최적화를 통해 직접 작성한 저수준 코드만큼 빠른 게이가 생성된다 |
| 3. 메모리를 전혀 사용하지 않고 프로그램을 실행할 수 있다 | 4. 컴파일 속도가 다른 언어에 비해 압도적으로 빠르다 |

오답입니다.

비용 없는 추상화는 고수준의 편리한 문법을 사용하더라도 런타임 오버헤드가 추가되지 않도록 컴파일러가 최적화하는 것을 뜻합니다. 오답 형성은 금전적 비용이나 컴파일 속도, 메모리 미사용 등으로 단어를 의미하는 것이요, 단합자의 이점을 이해해야 합니다.

학습 튜터

※자료 오버헤드(비중 없는 추상화)※는 교수진의 편리한 추상화 도구를 사용하더라도, 컴파일러가 이를 최적화하여 개발자가 직접 손으로 쓴 자수준 코드와 동일한 성능의 기계어를 만들어내는 것을 의미해요 [슬라이드 11]. 즉, 편리한 기능을 사용하더라도 실행 중에 추가적인 메모리나 CPU 자원 등의 런타임 오버헤드가 전혀 발생하지 않는다는 뜻이에요 [슬라이드 7] [슬라이드 11]

이해를 돕기 위해 자바와 러스트의 대조 예시를 들어볼게요.

자바: 제네릭을 사용할 때 런타임에 타입 소거를 거치며 박싱 오버헤드가 발생해요 [슬라이드 11].

러스트: 제네릭 함수를 정의하면 컴파일 타임에 타입별로 구체적인 함수를 생성하는 '단형화 과정'을 거치기 때문에, 런타임 오버헤드 없이 직접 호출이 실행돼요 [슬라이드 11].

다만, 리스트의 메모리 관리 방식이 모든 경우에 제로 오버헤드를 보장하지는 않는다는 한계도 함께 존재합니다 [슬라이드 3].

요약해줘

이번 강의는 Java 개발자의 관점에서 가... 나 버어에할 환경에서 동작하는 Rust의...

메모리 관리 방식의 혁신 (소유권과 빌려오기)

요약해줘 예제 더 헛갈려요

이 페이지 내용에 대해 물어보세요

0 - Q&A 루

강의를 보며 실시간 AI 질문 채팅

3

**풀고, 즉시 채점받고,
약점까지 짚어줍니다**

실전 모의고사

AI가 2023년 전 반년을 분석한 결과, 무제한 생성형 AI의 사용이 기업 생산성 향고 중시 결정, 생성형 AI 활용에 따른

98

파이썬에서 변수 할당을 올바르게 해석하기 위해 전제가 되는 기본 개념으로, 변수와 실제 데이터가 서로 독립적으로 존재함을 나타내는 기준은 무엇인가?

- 당첨지 0/15
- | | | | | |
|----|----|----|----|----|
| 1 | 2 | 3 | 4 | 5 |
| 6 | 7 | 8 | 9 | 10 |
| 11 | 12 | 13 | 14 | 15 |
- ☒ 일차 ☐ 2차 ☐ 3차
- 제출하기

1

20종 문제 유형으로 자동 출제

2

AI 총평 + 추약 단위 약점 리포트

재검 결과를 바탕으로 전 탈출 학습 진단아래로

이번 시험에서 75점 만점에 10점, F등급을 받았어요. 점수가 많이 아쉽지만, 어디서 흔들렸는지 정확히 짚어 두면 다음엔 분명 달라질 거예요.

[illegible]

오개념 그로blem 연습을 하거나 재연습하고, 개념/문제 예제의 특성 분류는 리스트와 함수에 id()를 직접 써서 비교하는 코드를 손으로 써 보세요. 앞은/뒤는 복사 분류는 종종 리스트를 copy()와 deepcopy()로 각각 복사한 뒤 내부 요소를 바꿔 보는 실습을 해 보세요. 기본 문자 형식 시퀀스 오개념은 def()로 번역 오류를 줄여잡아 주시길; 천천히 직접 복습해 보세요.

개념 하나하나를 코드로 확인하는 습관만 들이면 금방 따라잡을 수 있어요. 응원해!

3

객관식 정답 매칭 채점(AI) — 정확.즉시

문화법 제2조

각 문항의 점수와 채점 피드백을 확인하세요

전체 15문항 표시 중

문항 1 정답 EXAC

파이낸서 변수 할당을 올바르게 해석하기 위해 전제가 되는 기본 개념으로, 변수와 실제 데이터가 서로 독립적으로 존재함을 나타내는 기준은 무엇인가

틀린 문제만 보기

10-200

내역 4. 시정비사업 개입이 전

내역 4. 시정비사업 개입이 전

최도현 choidoheun@naver.com최도현 choidoheun@naver.com[illegible]

EXAC

EXAC

정답 E. 가변 객체는 내부 요

정답 E. 가변 객체는 내부 요

[illegible]

"문제 생성은 4개 에이전트 × 4 병렬 + 교차검증 자가교정(최대 3회)"

SYSTEM ARCHITECTURE

시스템 아키텍처

1 MSA로 인프라 분리

프론트(S3·CF) / 백엔드(Spring, FastAPI) / 데이터(RDS·S3)로 역할을 분리 구성
AWS 서술 리전에 Terraform(IaC)으로 전부 정의

2 프론트 (S3 · CloudFront)

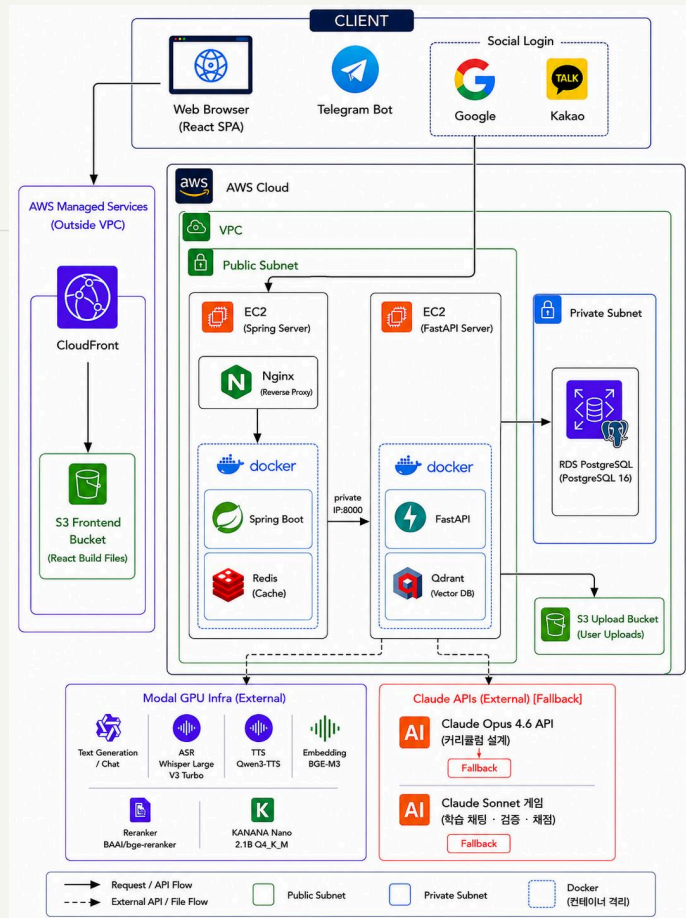
React 빌드 → S3(퍼블릭 차단, OAC로만 접근) → CloudFront CDN 배포

3 백엔드 (Spring, FastAPI)

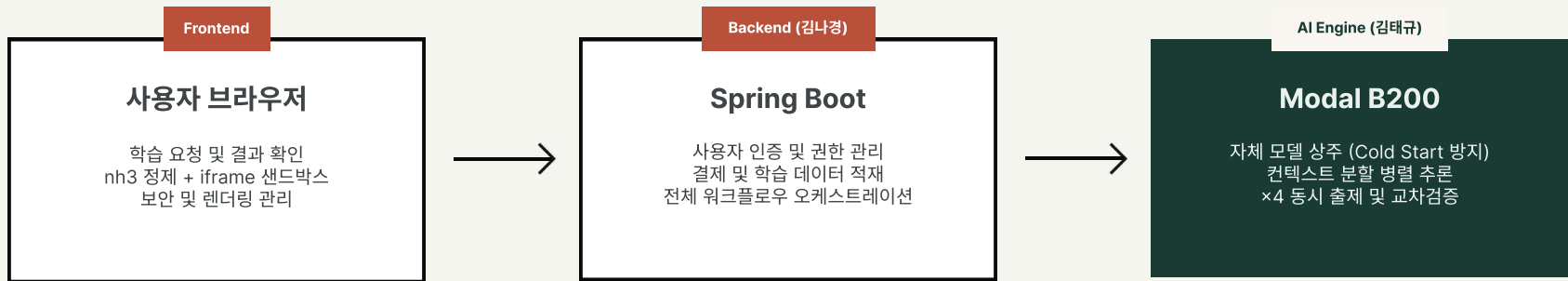
- Spring Boot(t3.micro): 메인 API + 내장 Nginx(80/443 진입점) + Redis
- FastAPI(t3.small): AI 추론 + Qdrant 벡터DB
- 외부는 Spring 80/443만 개방, FastAPI 8000은 Spring에서만 호출(최소 권한)

4 데이터 (PostgreSQL RDS · S3)

프라이빗 서브넷 격리, 외부 차단, EC2에서만 5432 허용, 암호화



외부 API 의존 없는 자체 모델 구동 인프라



"외부 LLM API 호출이 아니라 **Modal B200** 위 자체 모델 직접 구동 → 토큰 과금이 아닌 GPU 시간 과금으로 압도적 경제성(\$0.38) 달성"

CORE TECHNOLOGY

전문 에이전트들의 병렬 파이프라인

1

자료 분석

2

출제 계획

3

문제 생성

x4 병렬

4

오답 구성

5

정답 해설

6

교차 검증

초병렬 에이전트 협업

문제 생성 단계를 4개 에이전트가 동시 처리하여 생성 속도를 극대화하고, 검증 실패 시 자동 수정 루프가 작동합니다.

검증된 템플릿 라이브러리

16개 디자인 카테고리 와 20개 모의고사 문제 유형을 구조화하여 AI 답변의 품질 일관성을 완벽히 확보했습니다.

구조는 코드가 결정하고, AI는 오직 {{빈칸}}만 채웁니다

SLIDE TEMPLATE · 16종 중 1

ChapterStudio_V1 / concept_code

```
1 // ChapterStudio_V1 · concept_code (강의)
2 {
3   "slide_idx": 0,
4   "category": "code",
5   "title": "{{핵심_개념}}",
6   "narration": "{{설명_200~360자}}",
7   "visual": {
8     "type": "step_flow",
9     "data": "{{시각_데이터}}
10  },
11  "checkpoint": "{{성취도_체크}}
12 }
```

QUIZ TEMPLATE · 20종 중 1

ExamForge_V1 / ko_multiple_choice_5

```
1 // ExamForge_V1 · ko_multiple_choice_5 (5지선다)
2 {
3   "stem": "{{문항_지문}}",
4   "difficulty": "{{1~5}}",
5   "bloom_level": "{{이해}}",
6   "options": [
7     {"label": "1", "text": "{{보기_1}}", "is_correct": true},
8     {"label": "2", "text": "{{보기_2}}", "is_correct": false},
9     // ... 정확히 5개 · 정답 1개 ...
10  ]
11 }
```

구조는 코드가 **plan-first**로 고정(프레임·visual 9종·난이도) · AI는 **{{빈칸}}**만 →
강의 16종 · 문제 20종 항상 동일 품질, 추가 학습비 \$0

자체 인프라 최적화로 구현한 실측 데이터 기반의 압도적 속도

인프라 응답 최적화

초기 콜드 스타트	~12.4s
모델 사전 로드 시	~0.8s
API 오버헤드	0.1s 미만

"모델을 Modal 스토리지에 미리 로드해 콜드스타트를 줄이고, 사용자 체감 대기 시간을 최소화했습니다."

생성 시간 실측 비교

순차 처리 방식

462s



일과(ilgwa) 병렬 처리

120s

74%

생성 시간 감소 달성

45%

학습 준비 리드타임 단축

강의 1개 \$0.38, 어떻게 가능한가

강의 1개 생성 비용

\$0.38

미사용 시 유지비

\$0

STEP 01

자체 모델 상주

모델을 GPU에 상주시켜 로딩 오버헤드를 제거하고 즉시 생성합니다.

STEP 02

컨텍스트 분할

생성 작업을 독립 단위로 분할해 추론 효율을 높입니다.

STEP 03

동시 병렬 처리

단일 GPU에서 여러 태스크를 병렬 실행해 자원 활용을 극대화합니다.

"외부 LLM API가 아닌 GPU 사용 시간 기반 과금 → 그래서 \$0.38, 미사용 \$0"

EXPECTED IMPACT

데이터로 증명하는 실질적 교육 혁신 가치

\$0.38

혁신적 비용 절감

자체 모델 구동 메커니즘을 통해 기존 외부 API 대비 강의당 생성 비용을 획기적으로 낮추었습니다.

-76.8%

학습 준비 시간 단축

AI 에이전트 병렬 처리를 통해 커리큘럼 설계부터 문제 생성까지의 전체 비용과 시간을 단축했습니다.

~27%

 (추정)

학습 마진을 확보

운영 효율화 및 자동화를 통해 교육 서비스 제공자의 수익성을 개선할 수 있는 비즈니스 구조를 제시합니다.

1/100

 (추정)

교육 기회의 확대

압도적 경제성을 바탕으로, 누구나 최상위권 수준의 1:1 맞춤형 교육을 누릴 수 있는 교육을 목표로 합니다.

* \$0.38 및 -45%는 Modal B200 환경에서의 실제 측정값입니다.

EXPECTED IMPACT

데이터로 증명하는 실질적 교육 혁신 가치

비용 혁신

\$0.38

강의 1개당 생성 비용 실측치
(미사용 시 \$0 유지비)

지속 가능성

~27%

보수적 추정 마진을 확보로
안정적 서비스 운영 가능

생산성 혁신

-76.8%

최적화 전후 생성 시간 단축
(518초 → 120초)

교육 비용 감소

1/100

기존 사교육 대비 압도적으로
낮은 진입 장벽 형성

사람 강사

수일간의 준비 기간



일과 (ilgwa) AI

단 몇 분 만에 완성

CHALLENGES WE SOLVED

어려웠던 점과 해결한 방법

1

"AI는 매번 다르게 만든다"

김태규 · 설계와 AI

[문제] 같은 요청에도 형식·품질이 매번 달라 학습 자료로 사용 불가

[해결] 파인튜닝 대신 툴 고정(16디자인·20문제유형) → AI는 {{빈칸}}만 채움

2

"AI 생성 코드를 화면에 못 넣는다"

김태규 · 설계와 AI

[문제] AI 생성 HTML/CSS를 직접 삽입 시 XSS 등 보안 위험 발생

[해결] nh3 정제 + 정적 이미지 사전 렌더 + iframe 샌드박스 적용으로 위험 제거

3

"보안 그룹이 서로를 가리킨다"

김나경 · 배포(Terraform)

[문제] Spring ↔ FastAPI 양방향 통신이라 두 보안 그룹이 서로를 참조
→ inline 규칙이 순환(Cycle) 의존을 만들어 plan 단계에서 전체 실패

[해결] 한 방향(FastAPI → Redis)만 standalone aws_security_group_rule로 분리해
SG를 먼저 생성, 규칙이 나중에 양쪽 id를 참조하게 해 사이클 해소

4

"동시에 누르면 코인이 덜 차감된다"

김나경 · Spring/DB

[문제] 같은 시간에 동시 차감 요청이 겹쳐 read-modify-write 갱신 손실
→ 잔액 음수·이력 불일치 발생

[해결] 검증·차감을 한 문장에 담은 원자적 조건부 UPDATE(balance >= :amt)로 전환,
외부 I/O는 트랜잭션 밖으로 빼 락 구간 최소화

TEAM & GROWTH

기술적 한계를 돌파하며 함께 성장한 팀 '일과'

김태규

기여도 50%

설계 및 AI 파트

AI 에이전트 협업 파이프라인 설계 및 핵심 AI 로직 구현을 주도하며 시스템 아키텍처를 완성했습니다.

김나경

기여도 50%

Spring Boot 및 배포

안정적인 백엔드 인프라 구축과 자동화된 배포 시스템을 통해 서비스의 견고함을 확보했습니다.

기술적 성장

AI 에이전트 협업 설계 및 서버리스 비용 최적화 경험 확보

협업과 소통

역할 분리 및 효율적인 API 기반 협업 체계 준수

문제 해결

사용자 중심의 우선순위 설정 및 집요한 문제 해결 능력 체득

이과

igwa

맞춤 학습을, 누구나 부담 없이

맞춤형

저비용

확장성



감사합니다.